

INDIVIDUAL MODULE-WISE LAB WORKBOOK

EXPERIMENT 3

Module: Python BDD Restful Automations

Student Name: Puspendu Sekhar Das

Enrollment Number: 12023002022220

Department: CST

Institution: Institute of Engineering and Management, Newtown

Experiment: Python BDD Restful Automations

Module Name and Experiment Title

Module: Python BDD Restful Automations

Title: Behavior-Driven REST API Automation Using Python, Requests and Behave

Problem Statement

REST services expose application functionality through HTTP methods and structured responses. Manual API verification becomes repetitive when multiple requests, payloads, authentication conditions and response validations must be checked. The experiment demonstrates Python-based REST automation using the requests library and organizes scenarios using BDD/Behave and Gherkin concepts.

Objective

- Understand REST and SOAP concepts and the role of Python in API automation.
- Send GET, POST, PUT, PATCH and DELETE requests using the requests library.
- Validate status codes, headers and JSON response content.
- Handle authentication, cookies, sessions and reusable configuration.
- Organize API tests using Behave, Gherkin feature files, step definitions and hooks.
- Use scenario outlines, examples and tags for data-driven and selective execution.
- Understand reusable API BDD structure and Allure/reporting concepts.

Tools, Software and Concepts Used

Category	Details
Language	Python 3
HTTP Library	requests
BDD Framework	Behave
API Concepts	REST, HTTP methods, JSON payloads, headers and status codes
Automation Design	Feature files, step definitions, environment.py hooks and scenario outlines
Data / Configuration	Reusable payloads, configuration values, CSV/JSON or database-driven data
Reporting	Behave execution output and Allure reporting concept

Implementation Steps

- Install Python, requests and Behave and configure the project in the IDE.
- Create a reusable API configuration and send a GET request.
- Read the response object and validate status code, headers and JSON content.
- Automate POST/PUT/PATCH/DELETE requests with payloads and reusable headers.
- Use authentication, cookies and session management where required.
- Create a Gherkin feature file and implement matching step definitions.
- Use hooks, tags and scenario outlines to organize reusable BDD API scenarios.

Source Code / Representative Implementation

```
import requests

BASE_URL = "https://api.example.com"

def test_get_user():
    response = requests.get(BASE_URL + "/users/1")
    assert response.status_code == 200
    data = response.json()
    assert "id" in data
```

Behave Code

```
1 Feature: User API validation
2
3 Scenario: Retrieve a user
4     Given the API service is available
5     When I send a GET request for user "1"
6     Then the response status should be 200
7     And the response should contain an "id" field
```

Figure 1: Gherkin feature describing a REST API scenario

Python API Validation

```
import requests

response = requests.get("https://api.example.com/users/1")

assert response.status_code == 200
data = response.json()
assert "id" in data

print("GET request validated successfully")
```

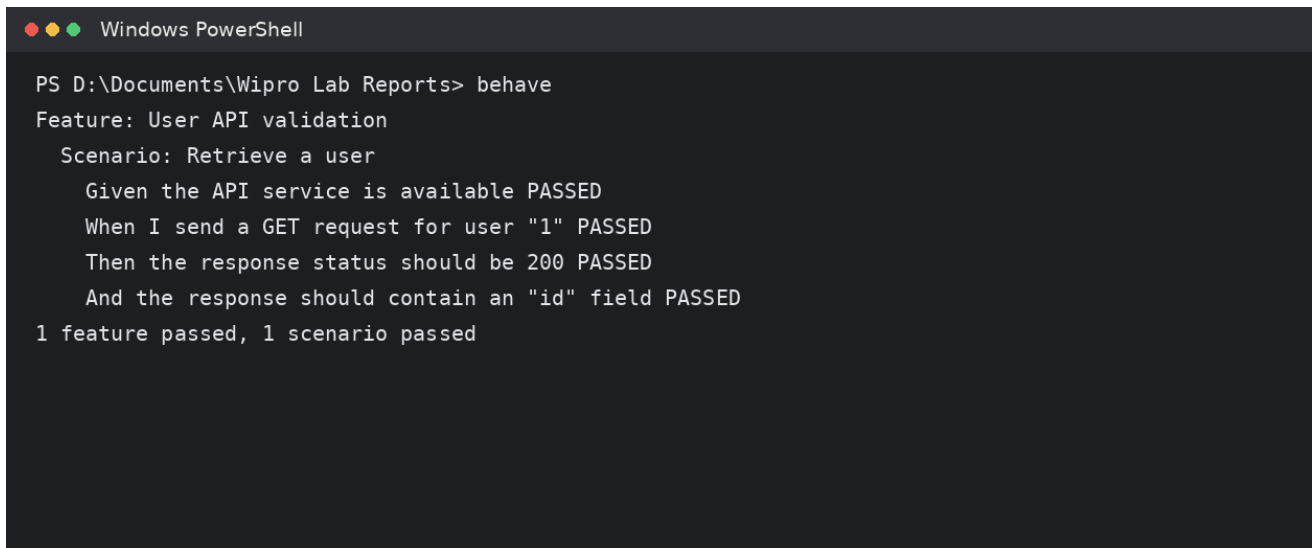
Behave Code

```
1 Feature: User API validation
2
3 Scenario: Retrieve a user
4     Given the API service is available
5     When I send a GET request for user "1"
6     Then the response status should be 200
7     And the response should contain an "id" field
```

Figure 2: Python requests implementation for REST response validation

Key Implementation Concepts

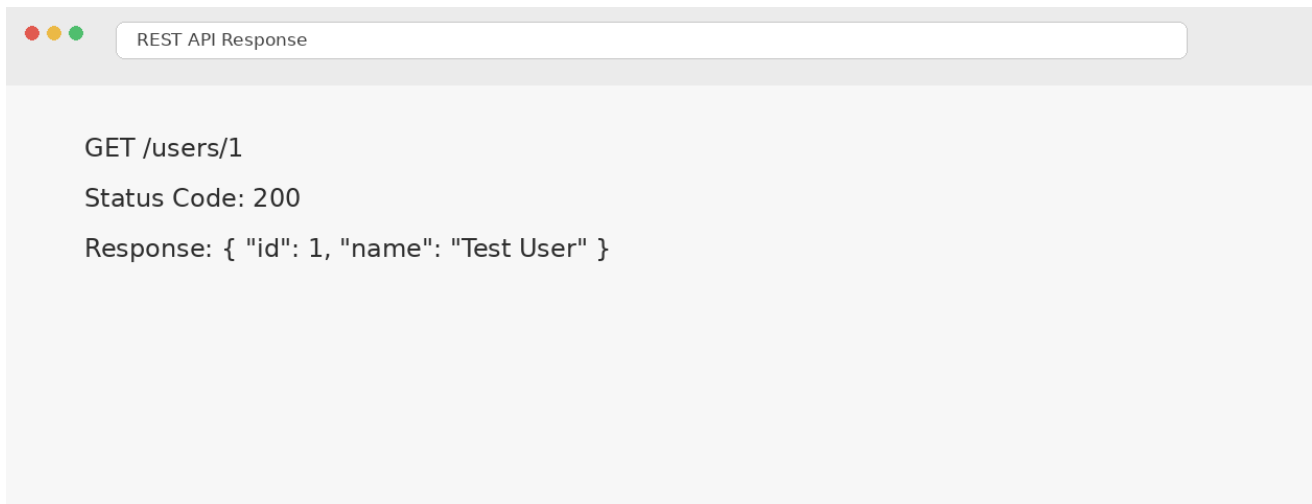
- GET retrieves resources, while POST creates data and PUT/PATCH update resources; DELETE removes resources.
- The requests response object provides status codes, headers, content and JSON data.
- Reusable headers, payloads and configuration values reduce duplication in API tests.
- Sessions can preserve cookies and connection information across multiple API calls.
- Behave uses Gherkin feature files to describe scenarios in business-readable language.
- Step definitions connect Gherkin steps to Python automation logic.
- environment.py can provide hooks for common setup and teardown.
- Scenario outlines and Examples support data-driven execution, while tags allow selected scenarios to run.
- Allure can be integrated as a reporting approach for BDD API automation.

A screenshot of a Windows PowerShell terminal window. The title bar shows three colored circles (red, yellow, green) and the text "Windows PowerShell". The command prompt shows the directory "PS D:\Documents\Wipro Lab Reports>" followed by the command "behave". The output displays a Gherkin feature and scenario: "Feature: User API validation", "Scenario: Retrieve a user", followed by four steps: "Given the API service is available PASSED", "When I send a GET request for user '1' PASSED", "Then the response status should be 200 PASSED", and "And the response should contain an 'id' field PASSED". The final summary line is "1 feature passed, 1 scenario passed".

```
PS D:\Documents\Wipro Lab Reports> behave
Feature: User API validation
  Scenario: Retrieve a user
    Given the API service is available PASSED
    When I send a GET request for user "1" PASSED
    Then the response status should be 200 PASSED
    And the response should contain an "id" field PASSED
1 feature passed, 1 scenario passed
```

Figure 3: Behave execution output for the REST API scenario

Output with Clearly Labelled Screenshots

A screenshot of a web browser window titled "REST API Response". The content area shows the details of a GET request to "/users/1". It lists the "Status Code: 200" and the "Response" as a JSON object: "{ 'id': 1, 'name': 'Test User' }".

```
GET /users/1
Status Code: 200
Response: { "id": 1, "name": "Test User" }
```

Figure 4: REST API response after GET request

Result

The REST API automation workflow was organized using Python requests and BDD/Behave concepts. API calls were represented as reusable scenarios, response status and JSON content were validated, and the execution flow was structured through feature files and step definitions.

Observations

- HTTP status codes provide an immediate indication of request success or failure.
- JSON responses can be parsed and validated directly using Python.
- Reusable sessions, headers and payloads simplify repeated API interactions.
- BDD feature files make API scenarios easier to read and organize.
- Hooks, tags and scenario outlines help create reusable and data-driven test suites.

Conclusion

The Python BDD Restful Automations module was completed through practical REST API and Behave concepts. The experiment demonstrated how Python can be used to automate HTTP interactions and how BDD structures those tests into readable, reusable scenarios.